Universitat Rovira i Virgili (URV) y Universitat Oberta de Catalunya (UOC)

Master in Computational and Mathematical Engineering

FINAL MASTER PROJECT

Area: YYY

# xxx title of the work xxx

## xxx subtitle (if it is the case) xxx

---

Autor: Complete name of the student

Tutor: Complete name of the director

---

Barcelona, March 12, 2026

Dr./Dra. (name) ....., certifies that the student (name) ..... has elaborated the work under his/her direction and he/she authorizes the presentation of this memory for its evaluation.

Director's signature:

# Credits/Copyright

A page with the specification of credits / copyright for the project (either application on one hand and documentation on the other, or unified), as well as the use of brands, products or services of third parties (including source codes). If a person other than the author collaborated in the project, his identity must be made explicit and what he did.

The most usual case is exemplified below, although it can be modified by any other alternative:

# FINAL PROJECT SHEET

| | |
|---|---|
| Title: | Descriptive of the project |
| Autor: | Name and surname(s) |
| Tutor: | Name and surname(s) |
| Date (mm/yyyy): | Date of delivery |
| Program: | Master in Computational and Mathematical Engineering |
| Area: | Name of the FMP area |
| Language: | English |
| Key words | Maxim 3 key words |

# Dedicatory / Quote

Brief words of dedicatory or quote.

# Acknowledgments

If deemed appropriate, mention the persons, companies or institutions that have contributed to the realization of this project.

# Abstract

Text with the synthesis of the project, that is, a text in which the definition of the project / problem addressed, its objectives / methods of resolution, and the results and conclusions are briefly explained (it can not be a list, but a text continuous written in a structured way). If it is necessary to put a reference in this text, it will be annotated at the bottom of the same page. In this section you can use a more literary and colloquial language than for the rest of the document.

In case of not writing the the document in English, it will be necessary to write the Abstract in duplicate. One of the versions must be textbf obligatorily in English. The other version can be written in Catalan or Spanish, in the language used for the rest of the report. The word Abstract will be changed to " textbf Resum " or " textbf Resumen " in the Catalan and Spanish versions, respectively.

Recommended length: 250 words maximum.

How to write a good abstract (in English):

http://www.ece.cmu.edu/ koopman/essays/abstract.html

**Keywords**: Work keywords separated by commas. For example for this document they could be Model, Guideline, Template, Memory, Final Master Project / Master

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction and Planning

## 1.1 Introduction

## 1.2 Motivation and Justification

## 1.3 Objectives

## 1.4 Project Planning and Management

The successful execution of the Final Master's Project (FMP), which focuses on developing a scientific C++ program for flood simulation, requires robust planning and diligent management. This course is weighted at 18 ECTS, equating to an estimated dedicated effort of approximately 450 hours. The timeline presented below serves as a guiding roadmap, designed to track progress against key project milestones and deliverables.

### 1.4.1 Work Methodology

The project methodology is structured around the four mandatory phases defined for the FMP, integrated with a phased software development approach. This hybrid methodology ensures that the core theoretical and design elements (Phases 1 & 2) are completed before critical implementation and validation (Phase 3) and final documentation (Phase 4). The iterative nature of the plan allows for necessary adjustments and includes dedicated time for thorough testing and writing throughout the academic year.

### 1.4.2 Project Schedule: Gantt Chart

The following Gantt chart (Figure 1.1) illustrates the proposed 24-week timeline for the project, distributing the estimated 450 working hours across the defined phases. Time buffers have been implicitly integrated into the task durations to account for unforeseen issues, necessary code refactoring, and

academic holidays.

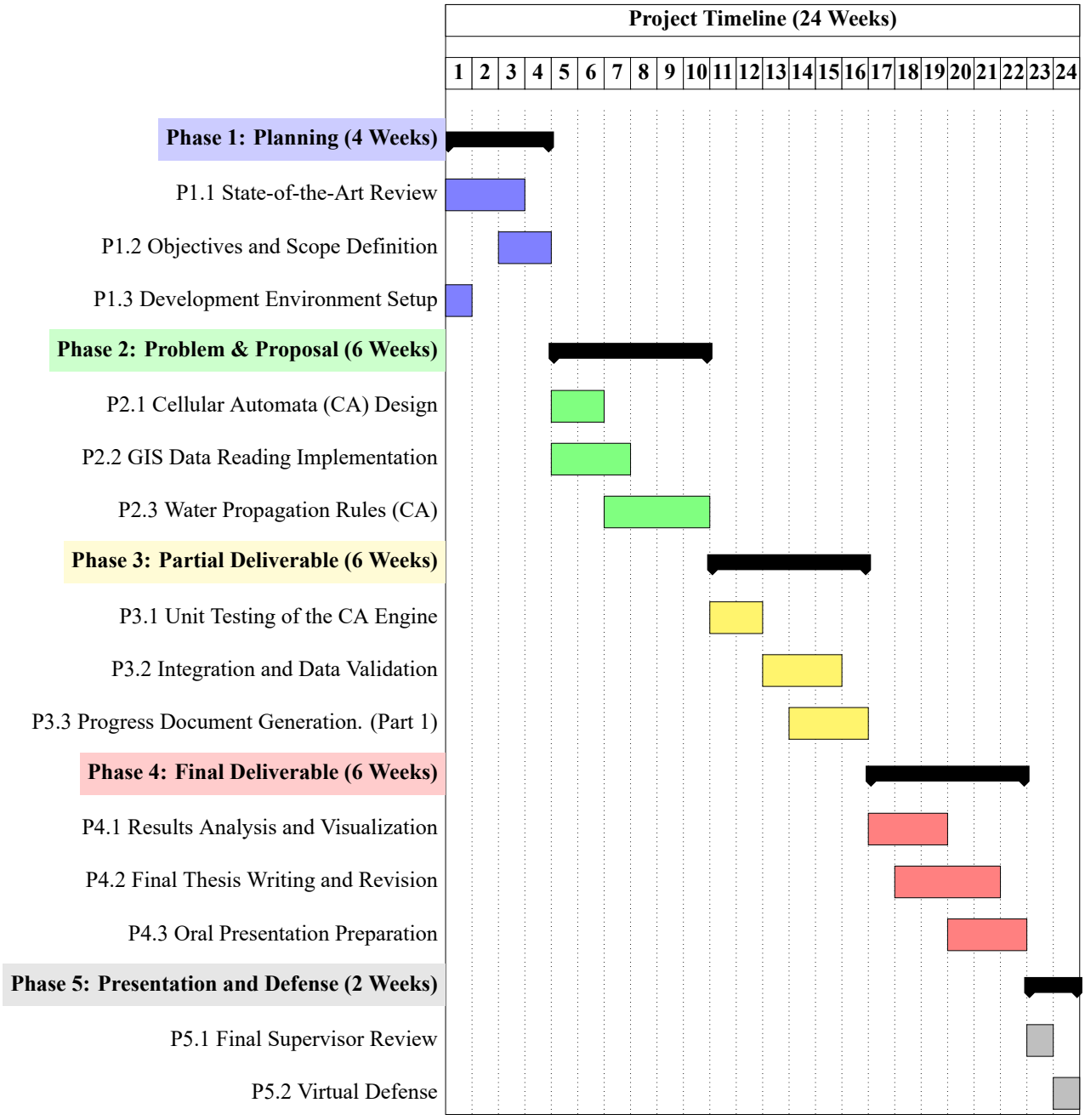


Figure 1.1: Project Timeline (24 Weeks)

### 1.4.3 Details of Activities

The following table provides a detailed breakdown of the tasks defined in the timeline, specifying the scope of work for each activity.

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

| ID | Activity | FMP Phase | Description (2-5 Lines) |
|---|---|---|---|
| **PHASE 1: PLANNING** | | | |
| P1.1 | State-of-the-Art Review | Phase 1 | Review existing scientific literature on flood simulation, focusing on Cellular Automata (CA) and C++ flow models. Key CA models suitable for water physics will be identified and the core references for the thesis will be collected. |
| P1.2 | Objectives and Scope Definition | Phase 1 | Clearly specify the flood scenarios the program will simulate (e.g., 2D vs. 3D, terrain type). Success criteria will be established, and the exact GIS data format required for input topography will be defined. |
| P1.3 | Development Environment Setup | Phase 1 | Install and configure the C++ compiler (including necessary GIS libraries like GDAL), the version control system (Git), and development tools. A basic LaTeX project structure will be created to initiate the project documentation. |
| **PHASE 2: PROBLEM DESCRIPTION AND PROPOSAL** | | | |
| P2.1 | Cellular Automata (CA) Design | Phase 2 | Define the transition rules and neighborhood structure for the CA (e.g., Moore or von Neumann). The C++ data structure will be designed to efficiently represent terrain cells and water depth. |
| P2.2 | GIS Data Reading Implementation | Phase 2 | Develop the C++ module for correctly reading and parsing the input GIS data (Digital Elevation Model or DEM). This module must initialize the initial state of the Cellular Automata based on the topography and the flood starting point. |

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

| ID | Activity | FMP Phase | Description (2-5 Lines) |
|---|---|---|---|
| P2.3 | Water Propagation Rules (CA) | Phase 2 | Code the core of the simulator, which includes the equations and logic for water transfer between cells in each iteration. This represents the underlying flow model that governs the flood simulation. |
| **PHASE 3: PARTIAL DELIVERABLE** | | | |
| P3.1 | Unit Testing of the CA Engine | Phase 3 | Conduct exhaustive testing of critical code functions, ensuring that GIS data reading, terrain initialization, and, mainly, the CA water propagation rules behave as expected. Focus will be on identifying precision and performance errors. |
| P3.2 | Integration and Data Validation | Phase 3 | Run the complete simulator with a representative set of GIS data. Simulation results will be compared against known flood data or other model outputs to validate the program's behavior. |
| P3.3 | Progress Document Generation (Part 1) | Phase 3 | Compile and draft the initial chapters of the FMP document in LaTeX. This includes the introduction, literature review, and a detailed description of the development methodology and the Cellular Automata design. |
| **PHASE 4: FINAL DELIVERABLE** | | | |
| P4.1 | Results Analysis and Visualization | Phase 4 | Process the simulation output data (depth maps, flooding time, etc.). External tools or C++ libraries will be used to generate clear graphs and visualizations that allow the interpretation of the flood impact. |
| P4.2 | Final Thesis Writing and Revision | Phase 4 | Complete the Results and Conclusions chapters of the LaTeX document. A full review of style, grammar, and formatting will be performed to ensure compliance with academic standards. |

Table 1.1: Detailed Breakdown of Final Master's Project Activities.

| ID | Activity | FMP Phase | Description (2-5 Lines) |
|---|---|---|---|
| P4.3 | Oral Presentation Preparation | Phase 4 | Design and create the presentation slides. The speech will be rehearsed, emphasizing the justification of the CA model, the C++ development process, and the key results obtained with the GIS data. |
| **PHASE 5: PRESENTATION AND VIRTUAL DEFENSE** | | | |
| P5.1 | Final Supervisor Review | Phase 5 | Final session with the FMP supervisor to address last-minute corrections to the document, code, and presentation. Minor details will be adjusted before the final submission and defense. |
| P5.2 | Virtual Defense | Phase 5 | Formal presentation of the FMP to the evaluation committee. Technical decisions, methodology, and project conclusions will be defended. |

## 1.5 Document Structure

# Chapter 2

# State of the Art

## 2.1 Current Approaches in Flood Modeling

### 2.1.1 Overview of Flood Simulation Paradigms

Flood modeling has undergone a significant transformation over the last decade, transitioning from static, historical-data-dependent assessments to dynamic, near-real-time predictive systems. This evolution is driven by the increasing frequency of extreme meteorological events, such as the catastrophic DANA (Depresión Aislada en Niveles Altos) experienced in the Spanish Mediterranean in October 2024 (Beltran et al., 2025; Muñoz et al., 2025). Current approaches can be broadly categorized into three main frameworks: physically-based hydrodynamic models, discrete computational models (such as Cellular Automata), and the emerging paradigm of Digital Twins within the context of Society 5.0.

### 2.1.2 Hydrodynamic Modeling and Shallow Water Equations (SWE)

The traditional ”gold standard” in flood simulation relies on the numerical solution of the Shallow Water Equations (SWE), which are derived from the Navier-Stokes equations under the assumption that the horizontal length scale is much greater than the vertical depth.

Advanced numerical methods, particularly Godunov-type schemes, have been extensively used to solve these equations on unstructured meshes. As noted by Aleksyuk and Belikov, 2017, these schemes are essential for handling complex topography with ”shoaling areas” and bottom discontinuities, where flow transitions between subcritical and supercritical regimes occur. These models use Riemann solvers to ensure shock-capturing capabilities, which is vital for simulating dam-break scenarios or violent flash floods in steep terrain.

Recent applications of these models, such as those discussed by Vasil’eva et al., 2021 in the Northern Caucasus, emphasize the integration of automated hydrometeorological networks. These systems allow for high-resolution simulations in small river basins where flash floods develop rapidly. However, while physically-based models offer high precision, their computational cost remains a significant bottleneck for large-scale or real-time applications, often requiring High-Performance Computing

(HPC) environments or GPU acceleration to become operational (Aleksyuk & Belikov, 2017).

### 2.1.3 Discrete Models: The Rise of Cellular Automata (CA)

As an alternative to the heavy computational requirements of differential equation solvers, Cellular Automata (CA) have emerged as a robust framework for flood inundation modeling. Unlike continuous models, CA operate on a discrete grid where each cell changes state based on local transition rules.

A significant milestone in this field was the development of the multilayer cellular automata ($m : n - CA^k$) by Fonseca et al., 2011. Originally applied to slab avalanche configurations, this model proved that complex geophysical phenomena could be effectively represented by separating the geographical data into distinct layers (GIS-based) and applying formal transition rules (represented via SDL - Specification and Description Language). This "Data-as-Interface" strategy allows for a clear decoupling between the model logic and the implementation, facilitating extensibility.

Current research in 2024 and 2025 has further refined CA-based solvers to include dynamic-wave approximations, achieving accuracies comparable to traditional Finite Volume methods but with execution speeds up to 300(Cellular Automata fast flood evaluation) have demonstrated the ability to predict maximum inundation depths in seconds, making them ideal for exploratory water system planning and real-time emergency response (Jamali et al., 2019).

### 2.1.4 Digital Twins and the Society 5.0 Framework

The most recent frontier in flood management is the integration of simulation models into Digital Twin (DT) architectures. Within the vision of Society 5.0, a Digital Twin is not merely a 3D visualization but a dynamic, real-time replica of the physical environment that ingests data from IoT sensors, satellite imagery, and meteorological radars (i Casas & i Palomes, 2026).

According to i Casas and i Palomes, 2026, Society 5.0 leverages Industry 4.0 technologies—such as Big Data, AI, and Digital Twins—not just for industrial efficiency but for human-centric social benefit, specifically in disaster resilience. In this context, a flood model becomes an "Open Model" that supports decision-making by allowing authorities to simulate "what-if" scenarios in real-time. For instance, during the 2024 DANA event, the lack of integrated real-time modeling was cited as a major hurdle in effective emergency management. The transition toward "Digital Twin Smart Cities" (DTSC) aims to bridge this gap by providing a continuous synchronization between the physical river basin and its digital counterpart (Wu & Mazzetto, 2024).

### 2.1.5 Data-Driven and Empirical Approaches

Complementing the numerical models are empirical approaches focused on peak flow estimation and hydrological response. Muñoz et al., 2025 highlight the importance of ”reconstructing” flood events using post-event field data and empirical formulas (such as the Manning equation and the rational method) to validate hydraulic models. Furthermore, the use of public datasets (AEMET, SAIH, Copernicus) has become standard for pre-processing inputs, although the ”spatialization” of rainfall remains a challenge in small, ungauged catchments (Vasil'eva et al., 2021).

### 2.1.6 Summary of the State of the Art

In summary, current approaches are moving toward a hybrid landscape where the mathematical rigor of Godunov-type schemes is being coupled with the computational efficiency of Cellular Automata. The trend for 2026 is clearly focused on:

1. **Heterogeneous Data Integration**: Using multilayer structures to handle static (topography) and dynamic (rainfall, IoT) data.
2. **Computational Efficiency**: Utilizing parallel architectures (CPU/GPU) to enable real-time forecasting.
3. **Holistic Decision Support**: Moving from isolated simulation tools to integrated Digital Twin platforms that follow the principles of Society 5.0.

## 2.2 Analysis of Shortfalls in Existing Solutions

Despite the significant advancements in hydrodynamic modeling and discrete simulation described in the previous section, the catastrophic flash flood events of 2024—most notably the DANA in the Júcar River Basin—have exposed critical limitations in the current state-of-the-art. These shortfalls can be categorized into four primary dimensions: computational latency, architectural rigidity, data integration silos, and the lack of a human-centric decision-making interface.

### 2.2.1 Computational Latency vs. Flash Flood Dynamics

The primary shortfall of physically-based models (SWE solvers) remains their execution time. As noted by Aleksyuk and Belikov, 2017, achieving high-precision results on complex, unstructured meshes requires high-performance computing resources that are often unavailable or too slow to deploy during the narrow ”lead time” of a flash flood. In events where the time from peak rainfall to peak flow is measured in minutes rather than hours, the computational overhead of traditional numerical

methods prevents them from being used as proactive forecasting tools. Instead, they are frequently relegated to post-event forensic analysis (Muñoz et al., 2025).

### 2.2.2 Monolithic Architectural Rigidity

Many existing flood simulation platforms are built as monolithic ”black boxes.” This architectural design makes it extremely difficult to extend the model or swap internal components (e.g., changing the infiltration algorithm or adding a new meteorological data source) without refactoring the entire system. This lack of modularity hinders the development of ”Open Models” as proposed in the Society 5.0 framework (i Casas & i Palomes, 2026). Furthermore, these systems are often tightly coupled with specific data formats, creating a barrier for interoperability between GIS tools and real-time simulation engines.

### 2.2.3 Simplified Representation of Spatial Heterogeneity

While standard Cellular Automata (CA) offer the speed required for real-time applications, they often suffer from an oversimplification of the physical environment. Most CA implementations treat the terrain as a single-dimensional grid, failing to represent the complex interactions between different layers of information—such as soil saturation, subterranean drainage systems, and varying urban roughness—simultaneously. The inability to handle ”multilayered” spatial data in a unified tensor-like structure limits the model’s capacity to simulate the non-linear behavior of water in urbanized basins (Fonseca et al., 2011).

### 2.2.4 The Socio-Technical Gap in Emergency Management

A final, critical shortfall is the disconnect between the technical output of simulation models and the operational needs of emergency responders. As highlighted by regarding the 2024 DANA crisis, the lack of a dynamic, real-time ”Digital Twin” meant that decision-makers were operating with fragmented and delayed information. Existing models do not easily integrate with the modern IoT and radar infrastructure needed to provide a continuous, synchronized digital representation of the evolving disaster.

## 2.3 Problem Statement

The core problem addressed by this thesis is the inability of current flood modeling frameworks to provide high-speed, physically accurate, and architecturally flexible simulations for real-time decision support in extreme flash flood scenarios.

Specifically, there is a technical gap between the high-precision but slow hydrodynamic solvers and the fast but overly simplified discrete models. This gap is exacerbated by the lack of a computational structure capable of managing the increasing complexity of heterogeneous geospatial data (static and dynamic layers) in a scalable manner.

The 2024 DANA event demonstrated that the traditional paradigm of ”predictive modeling” is insufficient for the demands of Society 5.0. What is required is a shift toward Reactive Digital Twins—systems that can ingest real-time data, process it through a multilayered transition logic, and output actionable results within seconds.

Consequently, this research seeks to solve the following technical challenges:

1. **Modeling Complexity**: How to formalize a multilayer cellular automata ($m : n - CA^k$) that can represent the interaction between diverse physical phenomena (precipitation, infiltration, runoff) as a multi-channel tensor.

2. **Architectural Decoupling**: How to design a software architecture (Hexagonal/Ports and Adapters) that allows the simulation engine to remain agnostic to specific GIS data formats and storage backends.

3. **Performance and Scalability**: How to implement this logic in a high-performance environment (C++) that leverages modern data structures (Tensors) to ensure the simulation speed remains well below the real-time threshold.

By addressing these issues, this work aims to provide a robust methodology for developing the next generation of flood management tools, capable of transforming raw environmental data into life-saving decisions.

# Chapter 3

# Methodology and Design

## 3.1 Proposed Analytical Framework and Resolution Strategy

To address the limitations identified in the current state-of-the-art—specifically computational latency, architectural rigidity, and the difficulty of integrating heterogeneous data—this research proposes a multi-dimensional analytical framework. The resolution strategy is built upon three fundamental pillars: the formalization of a multilayered discrete space, the adoption of a ”Data-as-Interface” philosophy, and the implementation of a decoupled software architecture.

### 3.1.1 The Multilayer Conceptualization: $m : n - CA^k$

The core of the proposed resolution lies in the adoption of the Multilayer Cellular Automata ($m : n - CA^k$) paradigm. Unlike standard Cellular Automata, which operate on a single state per cell, the $m : n - CA^k$ framework allows for the simultaneous representation of diverse physical and environmental variables across different computational planes.

In this framework, the river basin is modeled as a stack of $n$ information layers (e.g., topography, soil saturation, precipitation, and urban obstacles). This allows the system to handle the complexity of flash floods by defining interaction rules between layers (inter-layer) as well as within the same layer (intra-layer). By treating these layers as channels of a global tensor, the model can execute complex transitions—such as the infiltration of water from the surface layer to the soil layer—without losing the computational efficiency inherent to discrete models.

### 3.1.2 The ”Data-as-Interface” Strategy

A critical component of the resolution strategy is the ”Data-as-Interface” (DaI) approach. This methodology posits that the data structure itself should act as the contract between different system components. Following the principles of Society 5.0 and Open Models (i Casas & i Palomes, 2026), the proposed framework separates the formal specification of the model (what the water does) from its technical implementation (how it is programmed).

By using the IDRISI format and Hierarchical IDRISI Format (HIF), the framework ensures that both static GIS data and dynamic real-time sensor inputs are ingested through a standardized pipeline. This solves the ”data silo” problem and allows the simulation engine to remain agnostic to the source of the information, whether it comes from historical databases or live meteorological radar.

### 3.1.3 Architectural Resolution: Hexagonal Design

To solve the architectural rigidity found in monolithic simulation tools, this thesis proposes a Hexagonal Architecture (also known as Ports and Adapters). The resolution strategy involves isolating the ”Simulator Core”—which contains the $m : n - CA^k$ transition logic—from external dependencies such as file systems, GIS libraries, and user interfaces.

- **The Core**: Encapsulates the mathematical and physical logic of the cellular automata.
- **The Ports**: Define the required inputs (e.g., rainfall layers) and outputs (e.g., inundation maps).
- **The Adapters**: Handle the specific technical details of data conversion (e.g., Rasterio for GeoTIFFs or custom C++ loaders for HIF).

This decoupling ensures that the system is highly extensible. For instance, a new ”Infiltration Layer” based on a different physical formula can be added by simply creating a new adapter, without modifying the core simulation engine.

### 3.1.4 Computational Performance Strategy

Finally, the strategy addresses the latency problem by prioritizing ”Performance by Design”. The resolution involves implementing the simulation engine in C++, utilizing advanced data structures that map the multilayer cellular automata directly onto memory-efficient tensors. By representing the system as a multi-channel grid, the model can leverage vectorized operations and parallelism, aiming to achieve simulation speeds that allow for multiple ”what-if” scenario runs within the decision-making window of a real-world emergency.

### 3.1.5 Summary of the Resolution Approach

In summary, the proposed methodology does not merely provide a new algorithm but a complete technical framework:

1. **Analytically**, it uses $m : n - CA^k$ to manage environmental complexity.
2. **Architecturally**, it uses Hexagonal Design to ensure flexibility and ”Open Model” compliance.

3. **Technically**, it utilizes high-performance C++ and the IDRISI format to bridge the gap between GIS and real-time forecasting.

This integrated approach provides the necessary foundation to build a Reactive Digital Twin capable of responding to extreme events like the 2024 DANA with the speed and precision required by modern society.

## 3.2 Theoretical model: Multilayer Cellular Automata ($m : n - CA^k$)

### 3.2.1 Conceptual Foundation: The $m : n - CA^k$ Paradigm

To model the complex dynamics of flash floods, this research utilizes the Multilayer Cellular Automata ($m : n - CA^k$) framework as formalized by Fonseca et al., 2011. Traditional CA models are often limited by a single state per cell; however, the $m : n - CA^k$ generalization allows for a multidimensional representation of the environment.

In this model, the landscape is discretized into a grid where each cell is not a scalar value but a vector of states across $m$ layers. The parameter $k$ represents the number of main layers—those whose state is updated by a transition function in each time step—while the remaining $m - k$ layers provide the static or external environmental context.

- **Layer State ($E_m$)**: The state of a specific layer $m$ at a position $(x_1, \ldots, x_n)$ is defined by the function $E_m[x_1, \ldots, x_n]$.
- **Global State ($EG$)**: The global state of the automaton at a given georeferenced position is determined by the combination of states across all layers:

$$\Psi(E_1[x_1, \ldots, x_n], \ldots, E_m[x_1, \ldots, x_n]) = EG[x_1, \ldots, x_n]$$

### 3.2.2 Layer Specification and Computational Space

Following the $m : n - CA^k$ notation, this model is configured as follows:

- **Main Layers ($k = 1$)**: The only dynamic variable is the Water Depth ($H$). Its evolution is governed by the conservation of mass and the hydraulic gradient.
- **Secondary Layers**: These layers act as parameters for the transition rules. They include Elevation ($Z$), Manning's Roughness ($n$), and Rainfall Intensity ($R$).
- **Neighborhood ($V$)**: The model employs a Moore neighborhood, considering the 8 adjacent cells. This allows for an isotropic distribution of flow, essential for capturing the divergent nature of flash floods in complex topographies.

### 3.2.3 Specific Transition Equations

The state of the main layer $H$ at time $t+1$ for a cell $(i, j)$ is determined by the transition function $\Delta$, derived from the physical logic of water balance and potential energy minimization:

1. **Mass Accumulation (Rainfall)**: Initially, the water depth is increased by the net precipitation over the time step $\Delta t$:

$$H_{temp} = H_t + (R \cdot \Delta t)$$

2. **Hydraulic Head and Flux Calculation**: The flow is driven by the difference in the Hydraulic Head ($\eta = H + Z$) between the central cell and its neighbors. For each neighbor $n \in V_{Moore}$, the potential outflow $q_{out,n}$ is calculated:

$$q_{out,n} = \max(0, \eta_{central} - \eta_n) \cdot \Gamma$$

   Where $\Gamma$ is a transfer coefficient derived from the cell size ($\Delta x$) and roughness.

3. **Mass Conservation Scaling**: To maintain stability and prevent negative water depths, the total potential outflow is scaled if it exceeds the available water:

$$\sigma = \min\left(1.0, \frac{H_{temp}}{\sum q_{out,n} + \epsilon}\right)$$
$$q_{real\_out,n} = q_{out,n} \cdot \sigma$$

4. **Final State Update**: The new water depth is the result of the local balance between rainfall, total outflows to neighbors, and inflows received from neighbors:

$$H_{t+1} = H_{temp} - \sum_{n=1}^{8} q_{real\_out,n} + \sum_{n=1}^{8} q_{in,n}$$

## 3.3 System Macro-Architecture

The system is designed following a Loose Coupling strategy between two distinct technological domains, effectively separating ”Data Engineering” (preparation) from ”Computational Physics” (simulation). This decoupling is mediated by a standardized File-Based Interface, ensuring that the high-level data processing in Python and the high-performance execution in C++ remain independent yet perfectly synchronized.

### 3.3.1 The Data-as-Interface Strategy

Instead of integrating Python scripts directly into the C++ core via interpreters or APIs, the architecture utilizes a strictly defined File-Based Interface. The Data Pipeline acts as a ”Producer”, generating a structured repository of geographical and temporal data. The C++ Simulator acts as a ”Consumer”, ingesting these datasets through a dedicated InputAdapter.

The system is a unified environment based on the IDRISI standard. Static layers are stored as IDRISI Raster datasets (.doc/.img), while spatiotemporal data is managed through a custom Hierarchical IDRISI Format (HIF).

This strategy provides several key advantages:

1. **Format Homogeneity**: By using a single underlying raster standard (IDRISI), the simulator's input logic is simplified, reducing the overhead of managing multiple third-party libraries like HDF5 in the C++ environment.

2. **Technological Specialization**: Python manages complex GIS operations—such as coordinate reference system (CRS) transformations and remote sensing API calls—while the C++ core focuses exclusively on the numerical efficiency of Cellular Automata rules.

3. **Immutability and Reproducibility**: Once the pipeline execution is finalized, the input artifacts are fixed. This allows researchers to audit the exact state of the environment before a simulation run, ensuring that results are deterministic and reproducible.

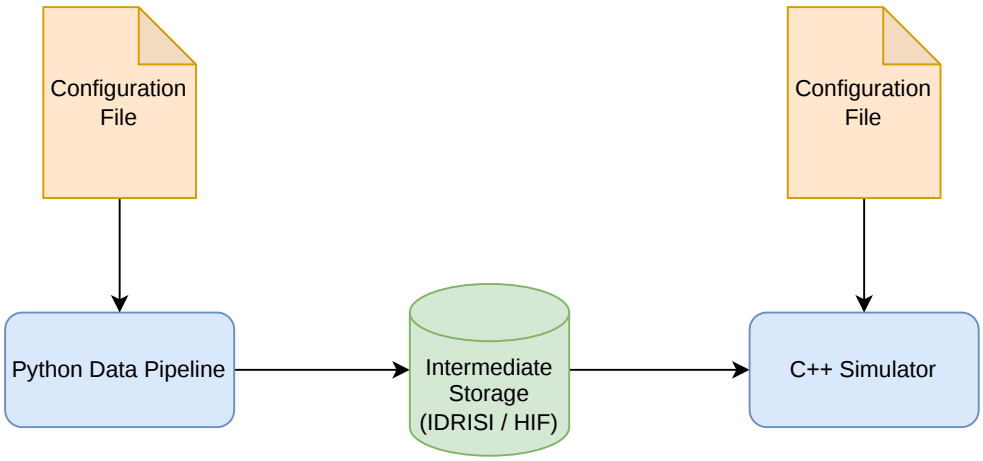


Figure 3.1: System architecture overview

## 3.4 The Hierarchical IDRISI Format (HIF)

To manage spatiotemporal data (e.g., hourly rainfall or dynamic water levels) without relying on complex binary containers like HDF5 or NetCDF, this project introduces the Hierarchical IDRISI Format (HIF). The HIF is a structural specification designed to store 3D data cubes using the native 2D IDRISI raster standard as a building block.

### 3.4.1 Conceptual Structure

The HIF treats time-series data as a discrete sequence of temporal frames. Each frame is stored as an independent IDRISI raster pair (.doc for metadata and .img for the data matrix), organized within a hierarchical directory tree. This allows for ”lazy loading” of simulation steps, where the engine only reads the specific frame required for the current time-step $t$.

### 3.4.2 Components of an HIF Dataset

An HIF dataset consists of three primary elements:

1. **Master Metadata (.hif)**: A global descriptor file that defines the spatial context (extent, resolution, and CRS) shared by all frames in the sequence. It serves as the entry point for the simulator.

2. **Temporal Attribute Manifest (attrs.json)**: A specialized metadata file that maps simulation time-steps to specific file paths. It contains the downgrade factors, ISO-8601 timestamps, and the ordered list of frame filenames.

3. **Frame Repository**: A flat collection of .img files, each representing a single point in time, without accompanying .doc files.

By adopting this format, the system achieves a balance between the simplicity of ASCII-based/flat-binary rasters and the requirement for organized, high-dimensional temporal data.

### 3.4.3 Physical Storage and Directory Topology

The physical realization of an HIF dataset is organized as a self-contained directory that encapsulates both the global spatiotemporal metadata and the individual raster frames. This structure is designed to facilitate modular access: the simulation engine can parse the top-level manifest to understand the temporal extent and subsequently perform random access to specific time-steps without the need to scan the entire dataset.

This tree topology illustrates the resulting hierarchy after the Data Pipeline exports a dynamic layer (e.g., rainfall).

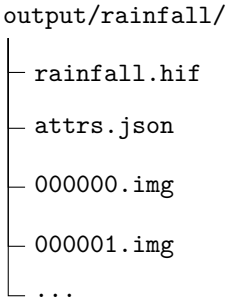


Figure 3.2: Hierarchical IDRISI Format (HIF) internal directory structure visualized as a tree topology.

## 3.5 Data Pipeline Design (Pre-processing)

The core of any flood simulation system lies in its ability to process vast amounts of geospatial data accurately and efficiently. The Data Pipeline is designed as a centralized orchestrator responsible for the ingestion, transformation, and provision of the physical parameters required by the hydraulic model.

The primary objective is to decouple data acquisition from the simulation logic. By doing so, the system ensures that changes in data formats or sources do not affect the stability of the simulator's core. This framework is built upon three fundamental pillars:

1. **Integrity**: Ensuring that spatial coordinates and resolutions remain consistent across all data layers.
2. **Scalability**: Allowing the inclusion of new physical variables (e.g., soil moisture or infiltration) without redesigning the existing architecture.
3. **Automation**: Minimizing manual intervention during the preprocessing of raster datasets.

### 3.5.1 Data Categorization: Static vs Dynamic Layers

To optimize processing and memory management, the methodology categorizes geospatial information into two distinct types based on their temporal behavior during a simulation:

- **Static Layers**: These represent physical characteristics that remain constant throughout the simulation event. Examples include the Digital Elevation Model (DEM) and Land Use (Roughness). These layers are processed once and serve as the immutable ”canvas” of the terrain.
- **Dynamic Layers**: Input data that informs the transition function.These involve variables that change over time, requiring a temporal dimension in their representation. Examples include Rainfall Intensity. The design must account for the sequential reading or generation of these files, ensuring that the simulator receives the correct ”snapshot” for each time step.

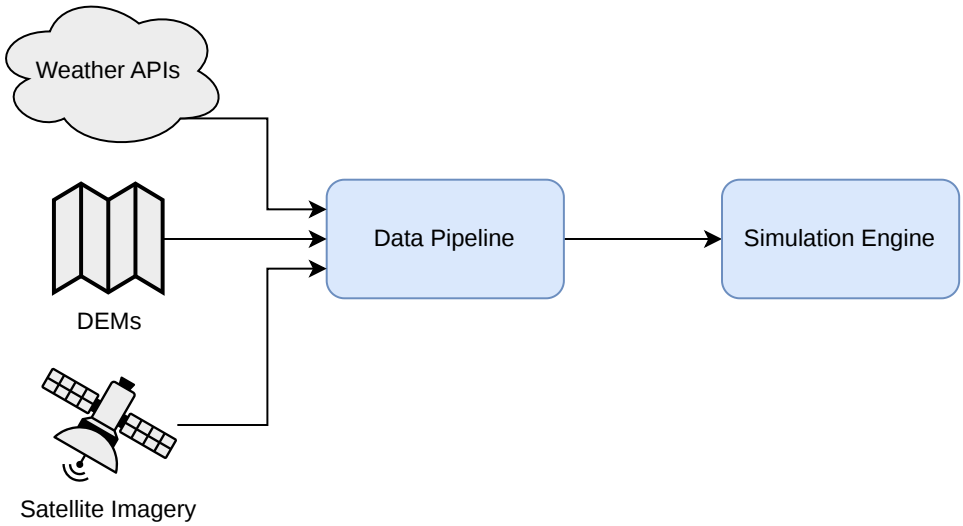


Figure 3.3: High-Level System Context

This conceptual separation allows the pipeline to apply different I/O strategies for each type, significantly improving the efficiency of the data flow.

### 3.5.2 Architectural Design Patterns

To achieve a balance between flexibility and rigorous data handling, the system architecture is grounded in three fundamental software engineering patterns: Abstraction, Inheritance, and the Registry Pattern.

#### 3.5.2.1 Abstraction and Encapsulation

The methodology treats every geospatial data source as a specialized object rather than a simple file path or a raw array. By abstracting the complexities of coordinate systems, pixel resolutions, and file formats into dedicated classes, the pipeline creates a unified interface. This encapsulation ensures that the rest of the simulator does not need to understand the internal structure of the data; it only interacts with high-level methods to retrieve physical values.

#### 3.5.2.2 Hierarchical Inheritance (Base Classes)

The design utilizes a hierarchical structure to manage the commonalities and differences between data types.

- **Base Components**: A set of abstract base classes defines the ”contract” that all specific layers and Input/Output (I/O) handlers must fulfill.

- **Specialization**: This allows the system to distinguish between the static nature of a Digital Elevation Model (DEM) and the time-varying nature of rainfall intensity, while still sharing the same underlying logic for spatial referencing and memory management.

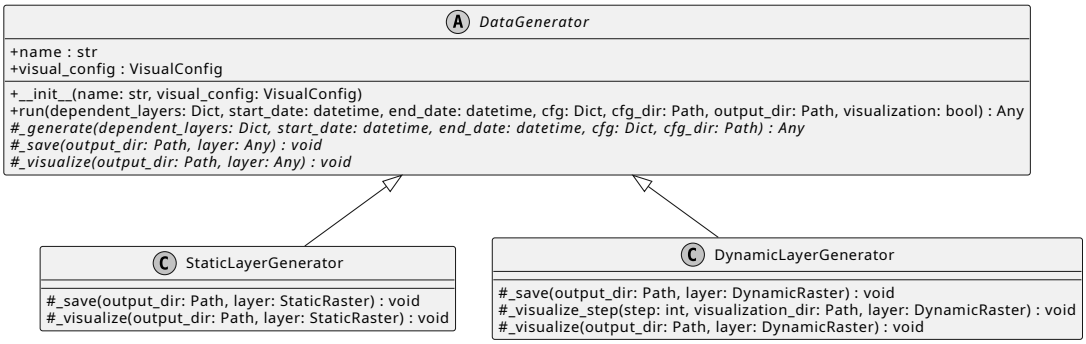


Figure 3.4: Data Generator Hierarchy

#### 3.5.2.3 The Registry Pattern for Extensibility

One of the most critical requirements for the TFM is the ability to integrate new physical variables (layers) without modifying the core orchestrator. To solve this, the methodology adopts a **Registry Pattern**.

In this approach, the Data Pipeline acts as a central hub that does not ”know” about specific layers (like Rainfall or Roughness) at compile-time. Instead, each new layer ”registers” itself into a dictionary-like structure. When the simulation starts, the pipeline simply iterates through this registry to initialize the required components. This design achieves Low Coupling, as adding a fifth or sixth layer only requires creating a new class that follows the established base contract, leaving the main execution logic untouched.

#### 3.5.2.4 Decoupling I/O from Logic

Finally, the architecture enforces a strict separation between Data Access (reading/writing files) and Data Logic (processing the raster values). By isolating I/O operations into dedicated I/O Handlers, the system becomes agnostic to the storage medium. If the project were to move from local IDRISI files to a cloud-based database or a different file format (like NetCDF), only the I/O handler would need to be updated, while the layer generation logic remains intact.

### 3.5.3 Data Flow Logic

The movement of data through the system is governed by a deterministic lifecycle, ensuring that the simulation engine receives a spatially and temporally synchronized ”snapshot” of the environment.

This flow is divided into four primary stages:

#### 3.5.3.1 Configuration-Driven Discovery

The process begins with the ingestion of a centralized configuration manifest (typically in YAML format), which serves as the ”Control Plane” for the entire data pipeline. Unlike a discovery-based system, the pipeline maintains a pre-defined registry of core physical layers (e.g., Elevation, Rainfall, Roughness) essential for the simulation. The configuration file is utilized to parameterize these layers and define the global execution context.

This stage manages three critical data dimensions:

1. **Global Simulation Metadata**: It establishes the temporal boundaries of the scenario (start and end timestamps) and the workspace topology (output directory and naming conventions). This ensures that all generated layers are synchronized within the same spatiotemporal window.

2. **Layer Parameterization**: For each architectural layer, the configuration provides specific execution parameters. This includes spatial resolutions, interpolation models (e.g., Variogram types for rainfall), and geographical constraints (bounding boxes). This allows the same pipeline logic to be reused across different study areas by simply adjusting the numerical constants in the config file.

3. **Visualization Manifest**: The pipeline identifies which layers require graphical rendering for research purposes. By defining a ”visualize” list in the configuration, the system selectively triggers the visualization engine post-generation, optimizing execution time by avoiding unnecessary plots for background layers.

By separating the layer definitions (hard-coded in the pipeline's logic for structural integrity) from their operational parameters (externalized in the configuration), the system achieves a high degree of reproducibility and scientific flexibility.

#### 3.5.3.2 Spatial Synchronization and Metadata Validation

Before any numerical processing occurs, the pipeline performs a Spatial Integrity Check. Flood simulations are highly sensitive to spatial misalignment. The flow logic enforces a ”Master Grid” approach, where secondary layers (such as rainfall or roughness) must be validated against the primary Elevation Layer (DEM).

- **Coordinate Reference System (CRS) Alignment**: Ensuring all data layers use the same projection.

- **Resolution Matching**: Verifying that pixel sizes are consistent across all rasters to avoid interpolation errors during the simulation.

- **Extent Clipping**: Ensuring that all dynamic inputs cover the exact bounding box of the static terrain.

#### 3.5.3.3 Sequential Execution: The Static-Dynamic Split

The data flow logic branches based on the temporal nature of the data:

1. **The Static Phase (Setup)**: The pipeline triggers the generation and storage of immutable layers (Elevation and Roughness). These are processed once and held in a ”Ready-to-Simulate” state.

2. **The Dynamic Loop (Runtime)**: For variables like Rainfall, the pipeline enters a cyclic flow. For every simulation time step, the logic dictates a ”Fetch-Transform-Provision” cycle, ensuring that the simulator always has access to the most recent temporal state without overloading the system’s memory.

### 3.5.4 Validation and Logging Strategy

The robustness of the data pipeline is sustained by a dual-layer strategy: rigorous pre-execution validation to ensure structural and spatial coherence, and comprehensive logging to provide full transparency during the transformation processes.

#### 3.5.4.1 Structural and Dependency Validation

Before processing any raster data, the system must validate the logical structure of the simulation request. This is approached using Graph Theory principles.

- **Directed Acyclic Graph (DAG)**: The dependencies between layers (e.g., Rainfall requiring an Elevation model for clipping) are treated as nodes and edges in a graph. The methodology requires a topological sort to ensure that no circular dependencies exist and that every component is initialized only when its requirements are met.

- **Schema Enforcement**: The configuration files are validated against a predefined schema to ensure that essential parameters, such as coordinate reference systems or simulation timeframes, are present and logically consistent (e.g., start_date must precede end_date).

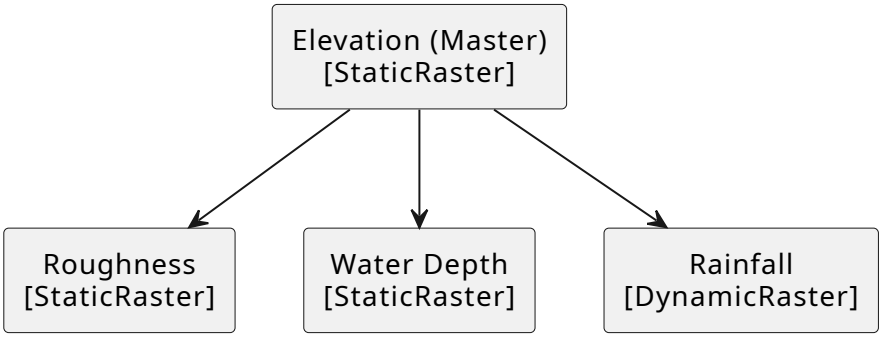


Figure 3.5: Layer Dependencies

#### 3.5.4.2 Spatial Integrity and Physical Constraints

The core of the validation strategy lies in ensuring ”Spatial Coherence”. Since the hydraulic model operates on a numerical grid, the pipeline enforces strict spatial constraints:

- **Domain Consistency**: Validating that all input datasets overlap with the requested study area.
- **Resolution Uniformity**: Detecting and flagging discrepancies in pixel size that could lead to significant calculation errors in the fluid dynamics solver.
- **No-Data Handling**: A theoretical framework for ”Nodata” values is established, ensuring that missing geographic information does not propagate as zero-values, which would lead to physical inaccuracies in the flood extent.

#### 3.5.4.3 Observability through Multi-Sink Logging

In long-running environmental simulations, observability is critical for both real-time monitoring and post-mortem analysis. The methodology adopts a Multi-Sink Logging Pattern:

1. **Transient Output (Standard Stream)**: Focused on high-level operational status (Success, Info, Error). It provides the user with immediate feedback on the pipeline’s progress without cluttering the interface with technical debt.
2. **Persistent Output (File-based Log)**: A detailed, low-level record of every transformation, I/O operation, and minor warning. This serves as a ”black box” for researchers to audit the data generation process.
3. **Severity Layering**: Messages are categorized by intensity (Trace, Debug, Info, Success, Warning, Error). This allows for granular filtering: while a researcher might only want to see successful milestones, a developer can access the ”Trace” level to inspect the exact metadata of a IDRISI being processed.

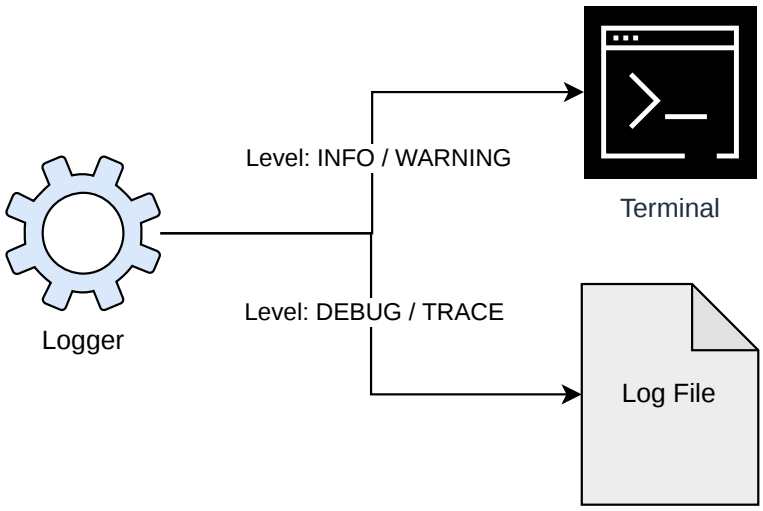


Figure 3.6: Logging Architecture

## 3.6 Simulation Engine Design

### 3.6.1 The Simulator Core: State inference logic

### 3.6.2 Hexagonal Architecture: Decoupling Using Ports and Adapters

### 3.6.3 Advantages of the design for physical layer extensibility

### 3.6.4 Data Structure and Tensor Model

### 3.6.5 Representation of the system as a multi-channel tensor

### 3.6.6 Layer Metadata Specification

# Chapter 4

# Implementation and Results

## 4.1 Software Environment and Technical Stack

### 4.1.1 Data Engineering Layer: Python Ecosystem

### 4.1.2 High-Performance Simulation Layer: C++ Environment

### 4.1.3 Geospatial Abstraction Libraries (Rasterio)

### 4.1.4 Spatio-temporal Data Management using HIF

### 4.1.5 Diagnostic and Observability Infrastructure

## 4.2 Implementation of the Data Pipeline

### 4.2.1 Core Architecture Implementation

#### 4.2.1.1 The DataPipeline Orchestrator

The DataPipeline class (implemented in pipeline.py) serves as the central engine of the system. Its implementation focuses on two main tasks: dependency resolution and execution management.

As seen in the source code, the pipeline initializes by loading a YAML configuration and registering available generators (Elevation, Rainfall, etc.) into a internal registry. The execution logic follows a rigorous flow:

1. **Dependency Discovery**: It builds an adjacency list where each layer identifies its prerequisites.

2. **Topological Sort**: Using graphlib.TopologicalSorter, the pipeline converts this list into a linear execution sequence. This ensures that the Elevation layer is always processed before any layer that requires spatial clipping.

3. **Context Injection**: During execution, the pipeline passes the output of previous layers (the dependent_layers dictionary) into the next one, allowing for seamless data transfer between components without global state variables.

#### 4.2.1.2 Base Classes and Structural Inheritance

To enforce the architectural patterns defined in Chapter 3, the implementation relies on a strict inheritance tree (found in base.py, static_layer.py, and dynamic_layer.py):

- **DataGenerator (Abstract Base Class)**: Standardizes the lifecycle of a layer through the run() method. This method acts as a template, orchestrating the generate, save, and visualize steps. This ensures that every layer, regardless of its content, follows the same operational protocol.
- **StaticLayerGenerator**: Specializes the base class for 2D data. It implements a standardized visualization routine using Matplotlib, including automated metadata boxes that display the Coordinate Reference System (CRS) and spatial resolution.
- **DynamicLayerGenerator**: Handles the added complexity of the temporal dimension. Its implementation includes logic for frame-by-frame visualization, effectively generating a temporal sequence of maps that can later be compiled into a video of the flood progression.

#### 4.2.1.3 Data Structures and Type Safety

The implementation uses Python dataclasses (in types.py) to define the internal data representation. By using SpatialContext, StaticRaster, and DynamicRaster, the system moves away from ”primitive obsession” (using simple arrays). This ensures that every tensor is always coupled with its spatial metadata (affine transform, CRS, and dimensions), preventing the spatial misalignment errors discussed in the methodology.

### 4.2.2 Input/Output Management

The Input/Output (I/O) subsystem is implemented through specialized handler classes that encapsulate the complexities of geographic file formats and directory structures. By isolating these operations in IdrisiIO and HierarchicalIdrisiIO, the pipeline ensures that the core generators remain agnostic to the underlying storage drivers.

#### 4.2.2.1 Static Raster Handling (IDRISI)

For immutable 2D layers such as elevation and roughness, the implementation relies on the IdrisiIO class. This handler manages the standard IDRISI dual-file specification:

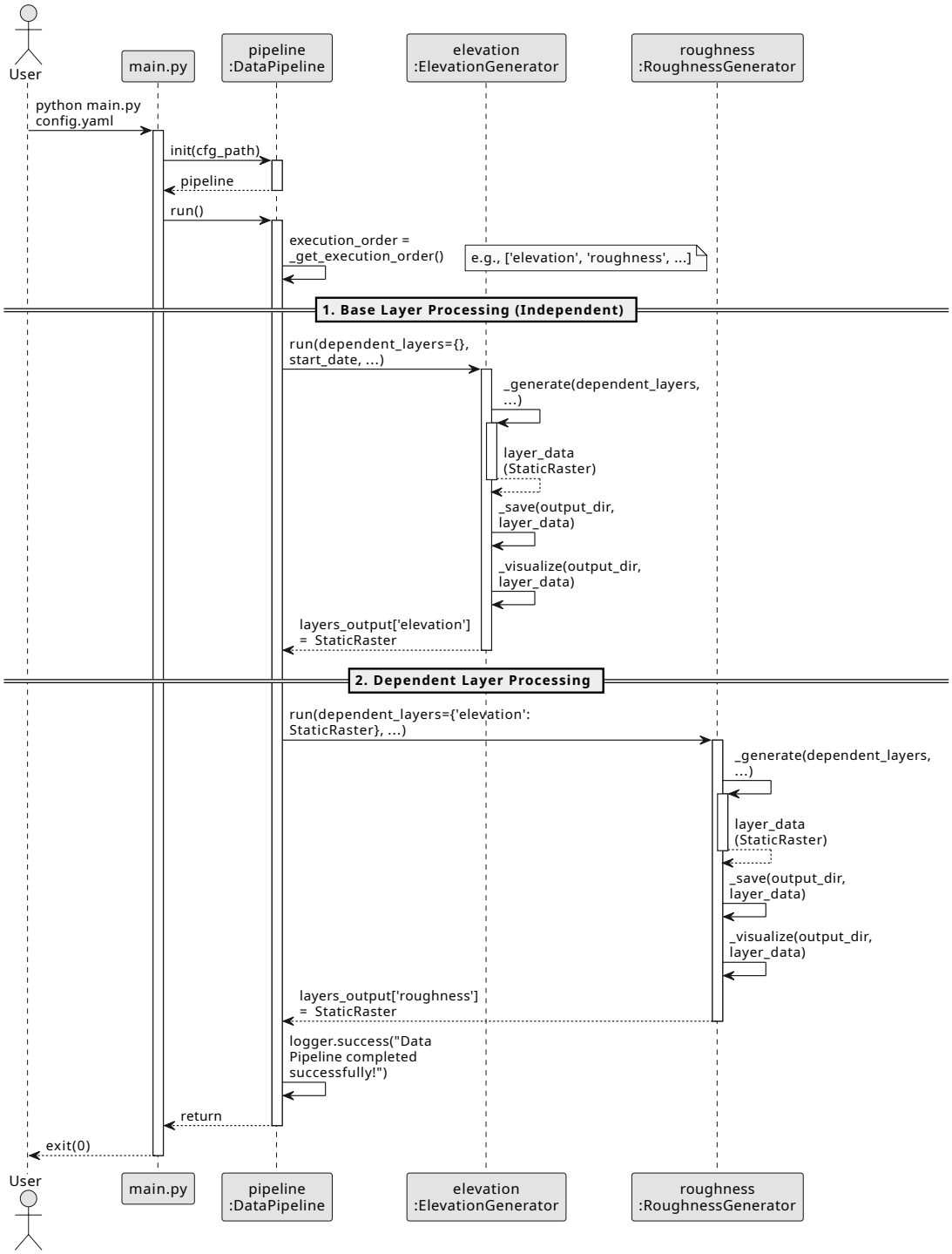


Figure 4.1: Sequence diagram of the *DataPipeline* execution flow. For visual simplicity, the diagram illustrates the iteration pattern over the first two layers of the simulator: an independent base layer (*Elevation*) and a dependent layer (*Roughness*), demonstrating the dependency injection mechanism at runtime. The rest of the layers inherit this same functional dynamic.

- **Metadata Serialization (.doc)**: The _write_doc() method programmatically generates a structured ASCII descriptor. It maps the internal SpatialContext (bounding box, CRS, cell size) to the specific fields required by the IDRISI standard, such as min. X, max. X, and posn. units.

- **Data Matrix Export (.img)**: The save() method handles the conversion of NumPy arrays into ASCII-encoded raster files. It employs a dynamic formatting strategy: integer-based layers (like land-use masks) are saved with zero decimal precision (using high-precision scientific notation to prevent data loss during the I/O process.

- **Directory Integrity**: The class automatically ensures the existence of target directories using pathlib, preventing runtime exceptions during the export phase of the pipeline.

#### 4.2.2.2 Dynamic Raster Handling (Hierarchical IDRISI Format - HIF)

To manage spatiotemporal data (e.g., rainfall time-series) without the overhead of heavy binary containers like HDF5, the system implements the Hierarchical IDRISI Format (HIF) through the HierarchicalIdrisiIO class. This format decomposes 3D data cubes ($Time \times Y \times X$) into a structured repository of standard IDRISI rasters.

- **Master Metadata (.hif)**: The implementation generates a global descriptor file with the .hif extension. This file mimics the standard IDRISI metadata but represents the spatial context of the entire temporal sequence, serving as the primary entry point for the simulation engine.

- **Temporal Manifest (attrs.json)**: A critical component of the HIF implementation is the generation of a JSON-encoded attribute file. This manifest maps each temporal step to its corresponding ISO-8601 timestamp and physical file path. This allows the simulator to perform "Temporal Seeking"—reading only the specific frames required for a given time-step without loading the entire dataset into memory.

- **Atomic Frame Persistence**: The save() method orchestrates a loop over the temporal dimension of the data cube. For each time-step $t$, it delegates the serialization of the 2D slice to the IdrisiIO handler. This ensures that every individual frame is a valid, independent IDRISI dataset.

- **Format Uniformity**: By leveraging the same underlying logic for both static and dynamic layers, the I/O subsystem minimizes the dependency on third-party libraries (like h5py or NetCDF4), simplifying the deployment of the C++ simulation core.

#### 4.2.2.3 Standardization of Spatial Metadata

Both I/O handlers enforce a unified spatial standard. During the save cycles, the pipeline ensures that all SpatialContext objects use the same coordinate units and orientation. This standardization is critical for the ”Spatial Synchronization” described in the methodology, as it prevents the simulation from running on misaligned grids.

### 4.2.3 Layer Generation Engines

The generation engines represent the ”concrete” implementation of the research methodology. Each engine is a Python class that inherits from either StaticLayerGenerator or DynamicLayerGenerator, ensuring that all physical layers provide a consistent set of outputs: a standardized numerical array, spatial metadata, and an automated visualization frame.

#### 4.2.3.1 Digital Elevation Model (DEM) Engine

The Digital Elevation Model (DEM) Engine is the foundational component of the data pipeline, as it establishes the primary topographic surface upon which the hydraulic simulation and other physical layers (such as roughness or water depth) are projected. This engine is implemented in the Elevation-Generator class, which automates the acquisition and standardization of terrain data.

**1. Data Sourcing and WCS Protocol**

The engine utilizes the Web Coverage Service (WCS) provided by the Instituto Geográfico Nacional (IGN). Unlike standard tiled map services (WMS/WMTS) which provide pre-rendered images, the WCS interface allows the pipeline to request raw numerical elevation values (MDT) for a specific geographic extent. The implementation targets the GetCoverage operation, specifying the output format as image/tiff to ensure high-bit-depth precision of the vertical data.

**2. Coordinate Transformation and Projection Logic**

A critical step in the implementation is the alignment between geographic input and the simulation’s projected grid. While the user provides coordinates in decimal degrees (WGS84 / EPSG:4326), the simulator requires a planar, metric system to perform accurate physical calculations.

The engine performs a transformation to the ETRS89 / UTM Zone 30N (EPSG:25830) coordinate system using the pyproj library. This process involves:

- Projecting the South-West and North-East corners of the requested bounding box.
- Calculating the exact dimensions of the study area in meters.

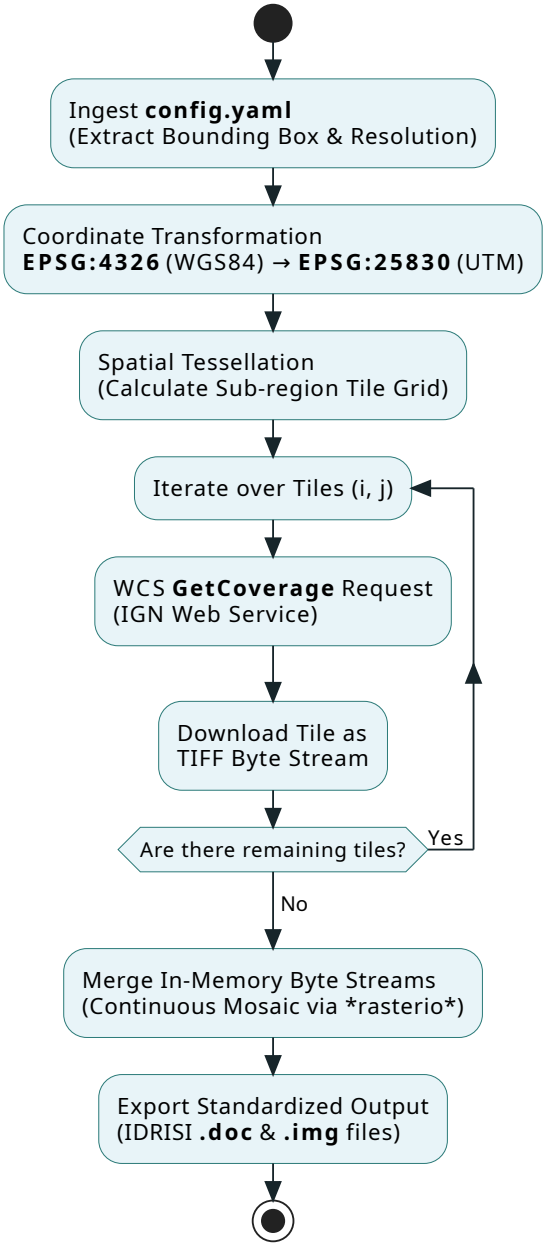


Figure 4.2: Flowchart illustrating the Digital Elevation Model (DEM) generation engine. The sequence details the pipeline execution from configuration ingestion and geographic projection mapping, through the WCS spatial tessellation strategy, to the final persistence in the IDRISI raster format.

- Computing the grid dimensions (width and height) based on the target resolution defined in the configuration file.

### 3. Payload Optimization through Tiling Strategy

Due to server-side constraints on the maximum payload size per request, the engine implements a Spatial Tessellation strategy. Large study areas are automatically decomposed into a discrete grid of tiles. The algorithm calculates the number of horizontal and vertical partitions $(i, j)$ and iterates through them, generating specific WCS subsets for each sub-region.

This iterative approach ensures:

1. **Resilience**: It prevents the IGN server from rejecting requests due to excessive data volume.

2. **Scalability**: It allows the processing of large-scale geographic regions that would otherwise exceed memory limits.

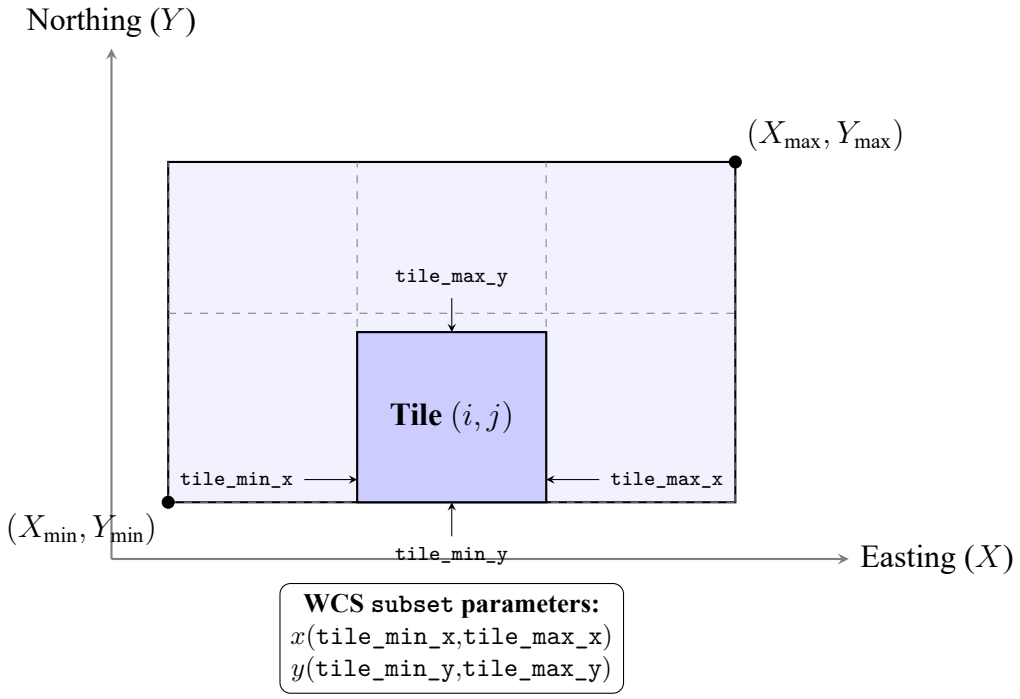


Figure 4.3: Visual representation of the WCS spatial tessellation strategy. The global bounding box of the study area is decomposed into a discrete grid. For each iteration, the engine requests a specific sub-region (Tile $i, j$) defined by its local coordinates, circumventing server-side payload limitations.

**4. Merging and Standardized Persistence**

Once all individual tiles are downloaded as in-memory byte streams, the engine delegates the reconstruction to the IdrisiIO handler. The tiles are merged into a continuous mosaic using rasterio, ensuring that overlapping edges and spatial metadata are correctly handled.

Finally, the resulting data is persisted using the standard IDRISI format (.doc and .img), where the elevation values are stored with floating-point precision to maintain the topographic integrity of the simulation environment.

#### 4.2.3.2 Roughness (Manning's $n$) Engine

This engine transforms land-use classifications into hydraulic roughness coefficients. It maps categorical data e.g., "urban area," "forest," "riverbed") into numerical values used by the transition functions of the Cellular Automata.

- **Type**: Static Layer.
- **Prerequisites**: Requires the Elevation layer to define the study extent.
- **Implementation Details**: [Placeholder: Details on the lookup table (LUT) logic and land-use data sources will be included upon script availability].

#### 4.2.3.3 Water Depth (Initial Conditions) Engine

The Water Depth engine defines the initial hydraulic state of the system. It is responsible for setting the starting water levels in riverbeds or floodplains before the first time step of the simulation.

- **Static (Initial State).**:
- **Defines the $t = 0$ state of the global tensor.**:
- **Implementation Details**: [Placeholder: Details on how initial water levels are derived or loaded from previous simulation states will be included upon script availability].

#### 4.2.3.4 Rainfall Intensity Engine

The Rainfall Intensity Engine is a critical component of the data pipeline, responsible for transforming discrete, asynchronous meteorological observations into a continuous spatiotemporal 3D tensor ($Time \times Y \times X$). This engine ensures that the flood simulator receives high-resolution precipitation forcing data, accounting for both the temporal evolution of storms and the geographic variations influenced by terrain.

The engine is architected following a modular approach, separating the concerns of data acquisition, spatial interpolation, and temporal management.

### 1. System Architecture and Orchestration

The orchestration of the rainfall data flow is managed by the RainfallGenerator class. This component acts as the high-level controller that integrates three distinct sub-systems:

1. **Extraction Layer (RainDataManager)**: Coordinates multiple data sources (e.g., SAIH Júcar) to fetch raw precipitation records.

2. **Interpolation Layer (KrigingInterpolator)**: Applies advanced geostatistical methods to generalize point data into a continuous surface.

3. **Integration Layer**: Conforms the resulting grids into the simulation's global grid structure, ensuring CRS (Coordinate Reference System) consistency and temporal alignment.

### 2. Multi-Source Data Acquisition

To ensure reliability and coverage, the engine employs a facade pattern through the RainDataManager. This allows the system to aggregate data from various meteorological networks simultaneously.

- **Standardized Fetching Interface**: All data sources implement a BaseFetcher interface, which enforces methods for station discovery, spatial filtering (within the simulation bounding box), and reprojection of coordinates to the target EPSG.

- **SAIH Júcar Integration**: A specialized fetcher (SAIHJucarFetcher) is implemented to scrape historical data from the "Confederación Hidrográfica del Júcar". This involves a complex process of parsing HTML for station metadata and extracting precipitation values from PDF reports using specialized parsing libraries.

- **Data Normalization**: Raw measurements, often provided in heterogeneous formats or accumulation periods, are normalized into RainDataPoint objects, representing intensity in mm/h at specific spatial coordinates.

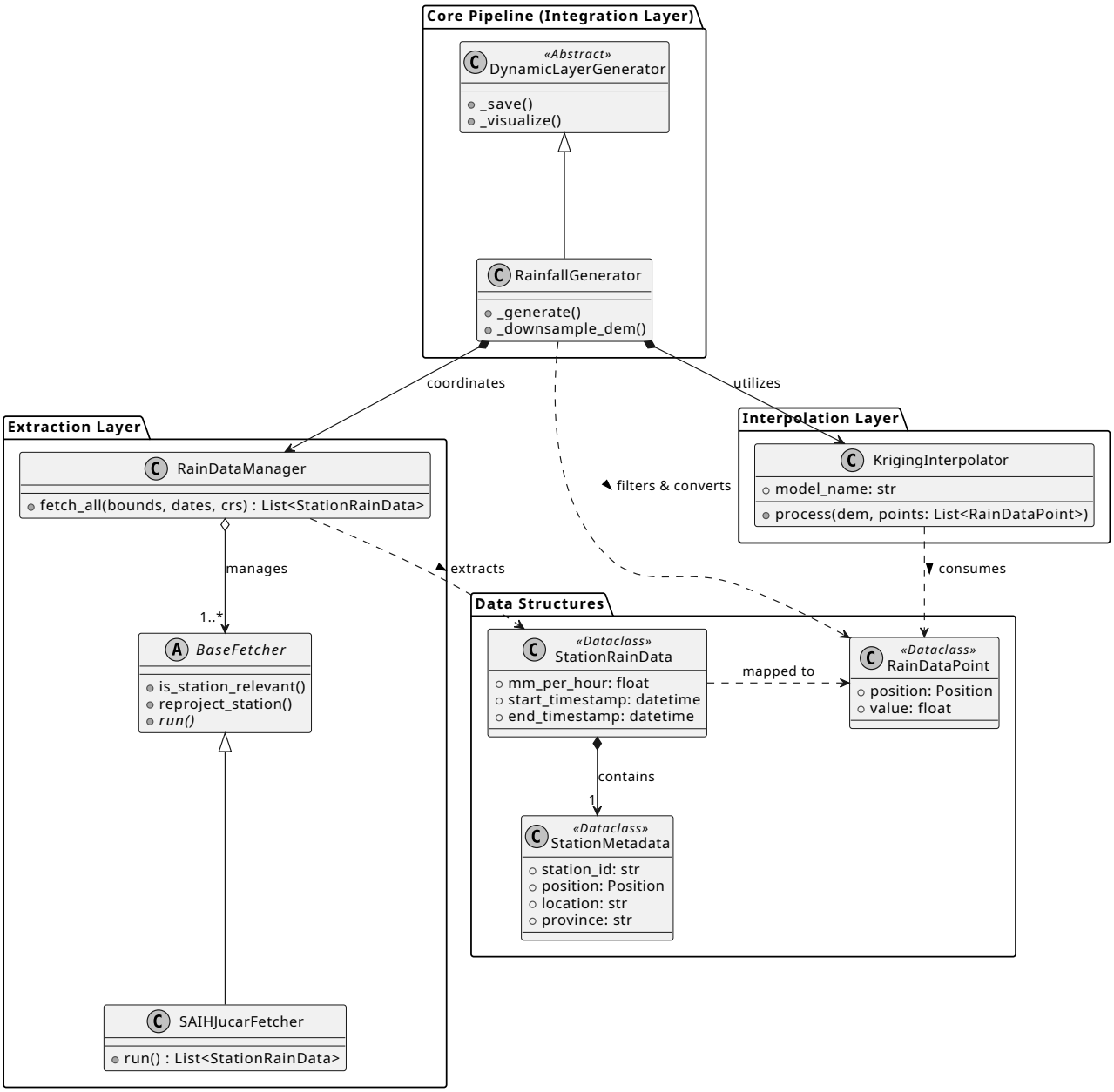


Figure 4.4: UML Class Diagram of the `RainfallGenerator` system architecture. The diagram depicts the structural composition of the module, highlighting the Separation of Concerns (SoC) across the Extraction, Interpolation, and Integration layers. Directional dashed arrows illustrate the flow of specific data structures (`StationRainData`, `RainDataPoint`) as they are fetched from external sources and processed into a unified spatial grid.

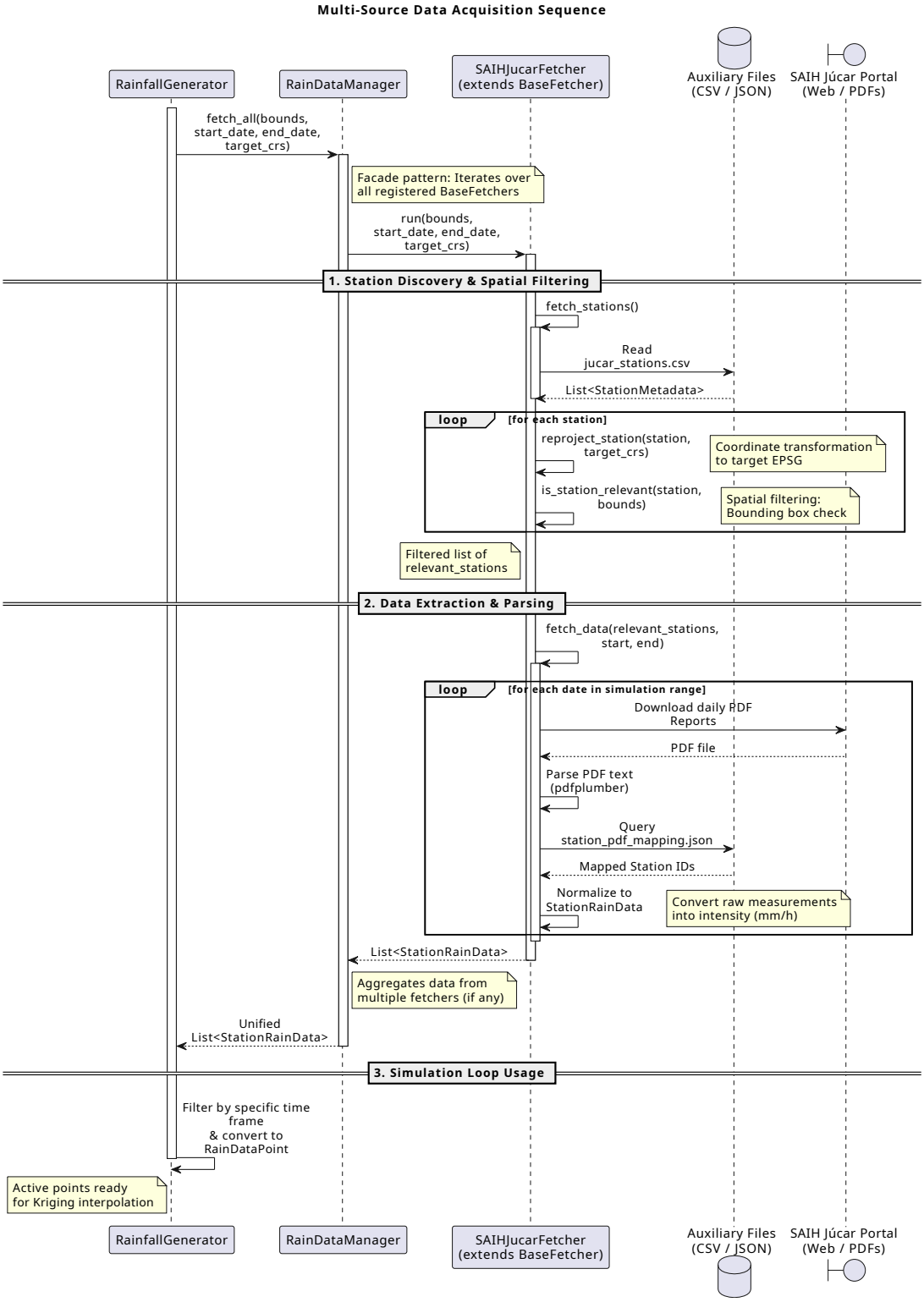


Figure 4.5: The process highlights the role of the `SAIHJucarFetcher` in station discovery, spatial bounding box filtering, coordinate reprojection, and the subsequent extraction and normalization of precipitation data from PDF reports.

### 3. Spatial Interpolation: Kriging with External Drift (KED)

The engine implements Kriging with External Drift (KED) via the KrigingInterpolator class. Standard interpolation methods (like IDW or Nearest Neighbor) often fail to capture the orographic effect—the phenomenon where higher elevations typically experience higher precipitation intensities. By using a Digital Elevation Model (DEM) as a secondary variable (drift), the engine produces meteorologically consistent surfaces.

The interpolation process follows a rigorous five-step geostatistical workflow:

- **Drift Estimation**: The system extracts elevation values from the DEM at each meteorological station's coordinates. It then fits a linear regression model to establish the relationship:

$$P_{trend} = \alpha + \beta \cdot Z$$

  where $P_{trend}$ is the predicted precipitation and $Z$ is the elevation.

- **Residual Calculation**: To isolate the spatial autocorrelation, the engine subtracts the elevation trend from the observed values at the stations:

$$R(s_i) = P(s_i) - (\alpha + \beta \cdot Z(s_i))$$

- **Variogram Modeling**: Using the gstools library, the engine calculates the experimental variogram of the residuals. It automatically fits a theoretical model (e.g., Spherical, Gaussian, or Exponential) to define the spatial structure, determining parameters such as the nugget, sill, and range.

- **Ordinary Kriging of Residuals**: The engine performs Ordinary Kriging on the residuals across the entire grid. This ensures that local deviations from the elevation trend are smoothly honors at the station locations.

- **Final Reconstruction**: The linear trend is reapplied to the kriged residual surface for every pixel in the grid:

$$P_{final}(x, y) = R_{krige}(x, y) + (\alpha + \beta \cdot Z(x, y))$$

### 4. Temporal Management and Tensor Assembly

The RainfallGenerator transforms the sequence of interpolated 2D grids into a structured 3D temporal tensor.

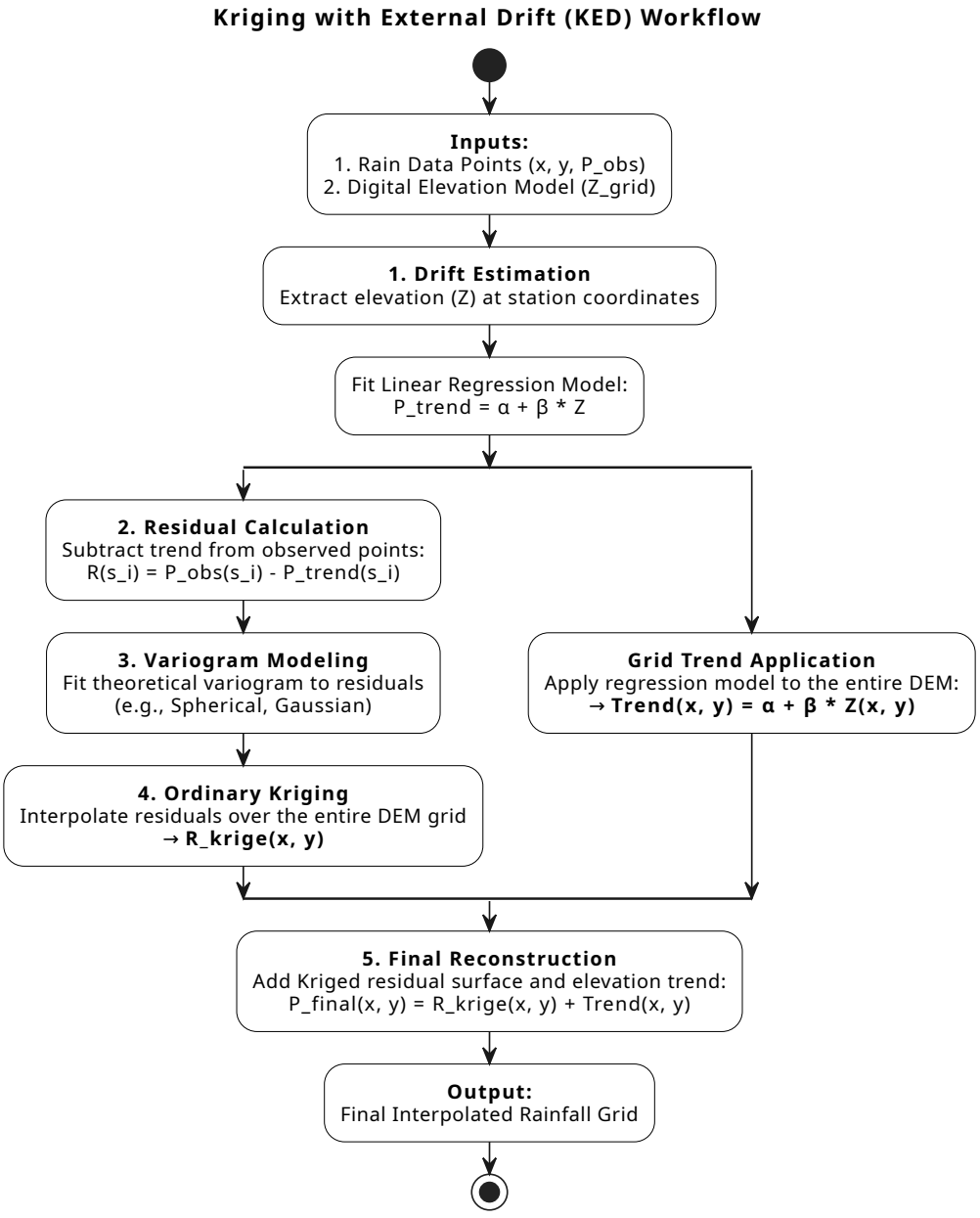


Figure 4.6: Flowchart of the Kriging with External Drift (KED) algorithm. The diagram illustrates the branching process where the global orographic trend is calculated across the entire DEM grid concurrently with the geostatistical interpolation (Ordinary Kriging) of the local residuals. Both branches converge in the final reconstruction step to produce the meteorologically consistent rainfall surface.

- **Temporal Resampling**: Since station observations and simulation time steps may not align, the engine performs temporal filtering. For each simulation frame, it identifies ”active” data points whose measurement windows cover the current timestamp.

- **Resolution Downsampling**: To optimize performance, the engine includes a _downsample_dem utility. This allows the Kriging process to run on a coarser version of the DEM if requested, significantly reducing the computational cost of the variogram fitting without losing the general topographic influence.

- **Data Integrity**: The engine handles edge cases, such as periods with insufficient active stations, by defaulting to zero-rainfall grids or logging warnings to prevent the simulation from receiving uninitialized memory.

### 4.2.4 Extensibility: Adding New Layers

The implementation’s modularity is best demonstrated by its extensibility. Adding a new physical variable (such as Soil Infiltration or Evapotranspiration) follows a strictly defined developer workflow, as dictated by the architecture in pipeline.py and base.py.

To integrate a new layer, the researcher must:

1. **Define a New Class**: Create a class inheriting from StaticLayerGenerator or DynamicLayerGenerator.

2. **Implement Logic**: Override the _generate method to define the physical transformation.

3. **Register the Component**: Add the class to the DataPipeline registry.

This workflow ensures that the new layer automatically inherits the ability to handle IDRISI/HIF I/O, error logging via Loguru, and automated visualization without writing a single line of extra infrastructure code.

### 4.2.5 Execution Results and Logging

The observability of the data pipeline is implemented as a multi-sink system that separates high-level operational status from low-level debugging information. This ensures that while the user receives clear progress updates, the full technical history is preserved for auditing.

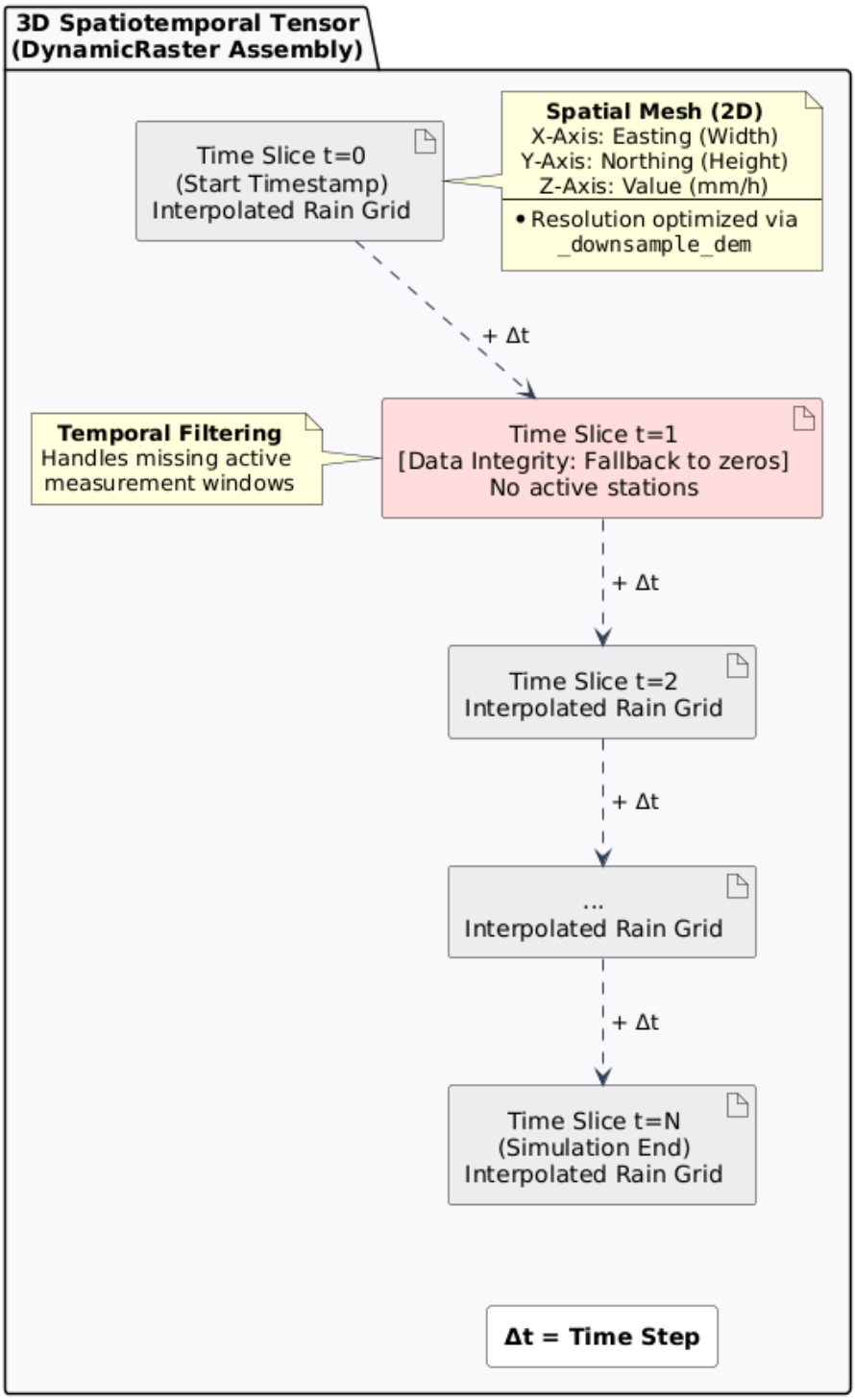


Figure 4.7: Conceptual representation of the 3D spatiotemporal data cube (`DynamicRaster`). Each 2D spatial mesh represents an interpolated rainfall grid at a specific time step ($\Delta t$). The chronological stacking illustrates the temporal filtering process, which maintains data integrity by inserting zero-rainfall grids when no active measurement windows are available. Spatial resolution can be optionally optimized via the `_downsample_dem` utility prior to tensor assembly.

#### 4.2.5.1 Dual-Sink Configuration

The logging system is initialized in the main.py entry point. It utilizes a filtered approach to manage the volume of information generated during the processing of large rasters:

- **Standard Output (Console)**: Configured with an INFO level threshold. It uses color-coded tags to highlight milestones (e.g., <green>SUCCESS</green> for completed layers). This sink provides a clean interface, showing only the pipeline initialization, the resolved execution order, and the final completion status.

- **Persistent File (log.txt)**: Configured with a TRACE level threshold. This file captures every granular detail, including the exact file paths being read, the affine transformation matrices extracted by Rasterio, and the specific memory addresses of the processed tensors.

#### 4.2.5.2 Runtime Traceability and Error Handling

The implementation ensures that any failure is not only caught but contextually enriched. As seen in the DataPipeline run loop:

1. **Contextual Breadcrumbs**: Each log entry includes the module name and line number (name:line), allowing developers to pinpoint exactly where a spatial misalignment or I/O error occurred.

2. **Exception Backtracing**: Through the logger.exception() method in the main try-except block, the system automatically captures and formats the full stack trace. This is crucial for debugging complex C++-linked errors that might otherwise be lost in a standard Python traceback.

#### 4.2.5.3 Output Directory Structure

Upon successful execution, the pipeline generates a standardized directory tree. This organization is critical for the C++ simulation core, which expects files in specific locations:

#### 4.2.5.4 Performance and Rotation

To prevent the logging system from becoming a bottleneck during high-frequency data generation, the file sink is implemented with asynchronous writing and rotation policies. The log files are rotated when they reach 10 MB and are automatically compressed into ZIP format, ensuring that the project's storage remains manageable even after hundreds of simulation runs.

```
output_simulation_YYYYMMDD/
├── logs/
│   └── log.txt                          # Full execution
├── elevation/
│   ├── elevation.doc                    # Base terrain (IDRISI)
│   ├── elevation.img
│   └── visualization/
│       └── elevation.png
├── roughness/
│   ├── roughness.doc                    # Manning's n (IDRISI)
│   ├── roughness.img
│   └── visualization/
│       └── roughness.png
├── rainfall/
│   ├── rainfall.hif                     # 3D Rainfall cube (HIF)
│   ├── attrs.json
│   ├── 000000.img
│   ├── 000001.img
│   ├── ...
│   └── visualization/
│       ├── rainfall_frame_001.png       # Temporal snapshots
│       └── ...
└── water_depth/
    ├── water_depth.doc                  # Initial hydraulic state (IDRISI)
    ├── water_depth.img
    └── visualization/
        └── water_depth.png
```

Figure 4.8: Standardized hierarchical directory structure generated by the data pipeline.

## 4.3 Implementation of the Simulator Core (C++)

### 4.3.1 Implementation of Hexagonal Architecture

#### 4.3.1.1 Input

**Multi-Format Raster File Input Adapter**

#### 4.3.1.2 Output

#### 4.3.1.3 State Updater

### 4.3.2 Algorithm for updating states using tensor operations

### 4.3.3 Performance optimization and parallelism

### 4.3.4 Tensor-based Grid Structure

### 4.3.5 The TemporalLayerManager: Efficient Memory Handling for Dynamic Data

## 4.4 Integration and Validation

# Chapter 5

# Conclusions and Future Work

## 5.1 Conclusions

## 5.2 Future Work

### 5.2.1 Integration of Meteorological Radar Data for Enhanced Rainfall Spatialization

While the current data pipeline successfully spatializes point-based rain gauge measurements using elevation-based drift (e.g., Kriging with External Drift), this methodology presents inherent limitations when modeling extreme weather events. Elevation acts as a static covariate that effectively represents long-term, climatological precipitation trends (the orographic effect). However, it cannot capture the highly dynamic, localized, and heterogeneous nature of real-time convective storm cells. Consequently, the interpolated rainfall tensor may misrepresent the exact geographical distribution of the precipitation nucleus during a specific simulation step.

To achieve professional-grade meteorological realism and improve the predictive accuracy of the hydraulic simulation, a primary line of future work is the integration of Meteorological Radar Data (such as the C-band and S-band radar networks operated by the Spanish Meteorological Agency, AEMET).

This integration would fundamentally upgrade the Rainfall Engine through the following technical implementations:

- **Radar Reflectivity Conversion**: Radar systems provide high-resolution spatial matrices of atmospheric reflectivity ($Z$). The pipeline would need to implement algorithms to dynamically fetch these grids and convert them into estimated rainfall intensity rates ($R$) using empirical formulas, such as the Marshall-Palmer relationship ($Z = aR^b$).

- **Radar-Rain Gauge Merging**: Because radar data is susceptible to systemic anomalies (e.g., beam blockage, signal attenuation, or overestimation due to hail), it cannot be used in isolation. The future pipeline should implement advanced merging techniques—such as Conditional

Merging (CM) or Regression Kriging. In this upgraded architecture, the radar imagery would serve as the highly detailed spatial covariate, while the physical rain gauges would provide the ”ground-truth” anchor points to correct the radar bias.

- **Enhanced Tensor Fidelity**: By replacing a static topographic covariate with a dynamic meteorological one, the resulting $Time \times Y \times X$ data cube would precisely reflect the trajectory, shape, and intensity of storm events.

Implementing this feature would drastically reduce spatial interpolation errors and provide the Cellular Automata ($m : n - CA^k$) core with a highly realistic input tensor, which is strictly necessary for the accurate modeling of rapid-onset flash floods.

# Bibliography


Aleksyuk, A. I., & Belikov, V. V. (2017). Simulation of shallow water flows with shoaling areas and bottom discontinuities. *Computational Mathematics and Mathematical Physics*, *57*, 318–339. https://doi.org/10.1134/S0965542517020026

Beltran, F. S., Boira, V., Vallés-Morán, F. J., & Viadel, R. C. (2025). *Revista fundada per joan fuster*. www.uv.es/lespill

Fonseca, P., Colls, M., & Casanovas, J. (2011). A novel model to predict a slab avalanche configuration using m:n-cak cellular automata. *Computers, Environment and Urban Systems*, *35*, 12–24. https://doi.org/10.1016/j.compenvurbsys.2010.07.002

i Casas, P. F., & i Palomes, X. P. (2026). Building society 5.0: A foundation for decision-making based on open models and digital twins. *Advanced Engineering Informatics*, *69*. https://doi.org/10.1016/j.aei.2025.103970

Jamali, B., Bach, P. M., Cunningham, L., & Deletic, A. (2019). A cellular automata fast flood evaluation (ca-ffé) model. *Water Resources Research*, *55*, 4936–4953. https://doi.org/10.1029/2018WR023679

Muñoz, R., Molner, J. V., Campillo-Tamarit, N., & Soria, J. (2025). Estimating peak flows in streams during the flash flood event of 29 october 2024 in spain: An empirical approach. *Water (Switzerland)*, *17*. https://doi.org/10.3390/w17213177

Vasil'eva, E. S., Belyakova, P. A., Aleksyuk, A. I., Selezneva, N. V., & Belikov, V. V. (2021). Simulating flash floods in small rivers of the northern caucasus with the use of data of automated hydrometeorological network. *Water Resources*, *48*, 182–193. https://doi.org/10.1134/S0097807821020160

Wu, W., & Mazzetto, S. (2024). A review of urban digital twins integration, challenges, and future directions in smart city development. *Sustainability 2024, Vol. 16, Page 8337*, *16*, 8337. https://doi.org/10.3390/su16198337